Wstęp do Uczenia Maszynowego Projekt – Raport

MAKSYMALIZACJA MOCY PREDYKCYJNEJ METODY KLASYFIKACJI

Krystian Kaliś

06.01.2025

1 Streszczenie

W pracy rozważono problem klasyfikacji do dwóch klas. Celem projektu było zaproponowanie metody klasyfikacji, dzięki której zbudujemy model o jak największej mocy predykcyjnej. Sztucznie wygenerowane dane zawierają zbiór testowy oraz treningowy. Nie został podany zbiór testowy zmiennej zawierającej klasy. Jako wynik działania zaimplementowanego programu otrzymano wektor prawdopodobieństw przynależności do klasy 1 dla obserwacji ze zbioru testowego. Na początku dane zostały mechanicznie sprawdzone. Później został zamodelowany Pipeline, w którym przygotowano dane, wykonano ich skalowanie do rozkładu normalnego, usunięto stałe zmienne, wyselekcjonowano najważniejsze z nich, a także usunięto zmienne nieskorelowane ze zmienną przechowującą klasy 1 i 2. Następnie zbudowano modele. Dokładność modeli była mierzona za pomocą miary zrównoważonej dokładności. Wybrano takie hiperparametry, które maksymalizują tę wielkość. Na wybranych, optymalnych modelach wykonano Voting oraz Stacking.

2 Wstępne przygotowanie zbioru danych

Pracę rozpoczęto od wczytania zbiorów danych z plików $artifical_train_data.csv$, $artifical_train_labels.csv$ oraz $artifical_test_data.csv$. Kolejno przypisano je do zmiennych X_train, y_train oraz X_test. X_train jest macierzą cech, która zawiera 1500 wierszy oraz 100 kolumn reprezentujących cechy. X_test ma tyle samo kolumn oraz 500 wierszy, natomiast y_train to wektor zmiennej docelowej, zawierający 1500 obserwacji i jedną kolumnę z klasami 1 oraz 2. y_train to wynik, który mamy przewidzieć na podstawie zmiennych z X_train. Liczba wierszy dla odpowiednich zmiennych zgadza się. Nie ma żadnych braków danych. Wszystkie zmienne są numeryczne, mają wartości typu "int". Wyrysowujemy histogramy dla zmiennych reprezentowanych jako kolumny w X_train oraz X_test by zbadać ich rozkłady. Możemy zauważyć, że zmienne przyjmują wartości z podobnych przedziałów liczbowych. Obserwacje odstające nie są widoczne na histogramach, większość z nich jest symetryczna i wycentrowana. W zmiennej y_train mamy 755 obserwacji będących jedynkami oraz 745 dwójek. Możemy więc stwierdzić, że zmienna y_train jest bardzo dobrze zbalansowana. Na tym kończymy wstępne przygotowanie danych, reszta tego procesu została wbudowana w Pipeline'y.

3 Właściwe przygotowanie zbioru danych

Obróbkę danych wykonano z wykorzystaniem narzędzia Pipeline, które zapewnia rzetelność wyników i pozwala uniknąć zjawiska wycieku danych. Pipeline przyjmuje listę krotek, każda z nich ma nadaną nazwę oraz metodę, jaka jest wykonywana na danych. Na początku skalujemy dane treningowe tak, że otrzymujemy dane, dla których średnia wynosi 0, a odchylenie standardowe to 1. Do tego celu wykorzystujemy transformator $StandardScaler()$. Z danych treningowych usuwamy te zmienne, których wariancja po przeskalowaniu nie przekracza wartości 0.05. Służy ku temu metoda $VarianceThreshold(threshold = 0.05)$. Danych nie uzupełniamy, ponieważ nie ma braków danych. Po wytrenowaniu takiego Pipeline'u z dwoma estymatorami na danych treningowych okazuje się, że żadna ze zmiennych nie ma niskiej wariancji, zatem ilość zmiennych nie ulega zmniejszeniu się.

Kolejną zasadniczą metodą wykorzystaną w Pipeline'ach jest selekcja zmiennych. Zaproponowano kilka sposobów jej wykonania. W celu identyfikacji nieliniowych zależności między zmiennymi objaśniającymi a zmienną celu, zastosowano metodę selekcji cech opartą na Informacji Wzajemnej (Mutual Information). W przeciwieństwie do standardowej korelacji Pearsona, metoda ta pozwala wykryć złożone relacje, co w przypadku sztucznie generowanego zbioru danych może być kluczowe dla jakości predykcji. Jako parametr przyjmuje ona liczbę zmiennych jakie mają pozostać w zbiorze danych, wykorzystane wielkości tego parametru to $k \in \{20, 40, 50, 60, 80, 90\}$. Metoda Informacji Wzajemnej korzystająca z entropii opiera się na ocenie każdej cechy z osobna. Inna metoda selekcji zmiennych: $SelectFromModel()$ uwzględnia zaś zespoły cech, a nie jedynie poszczególne cechy z osobna. Transformator ten wykorzystuje informację z modeli posiadających metody $feature_importance_$ lub $coef_$. Modele zaaplikowane do tej metody to regresja logistyczna z regularyzacją lasso i maksymalną liczbą iteracji wynoszącą 5000 oraz model lasu losowego z różnymi liczbami budowanych drzew 50 lub 100. Ważność cech w modelu jest określana na podstawie liczenia średniej bądź mediany. Optymalną liczbę cech po selekcji zmiennych dobrano przy etapie budowy modeli w procesie walidacji krzyżowej, o czym będzie mowa w dalszej części pracy. Spośród wymienionych metod selekcji do każdego modelu wybrano jedną, najlepiej szacującą wartość zrównoważonej dokładności. Wybór większej ilości metod selekcji jednorazowo jest bezsensowny ze względu na różną specyfikę tych metod. Przykładowo jeśli mamy dwie cechy, które same w sobie są słabe, to prawdopodobnie wypadną ze zbioru danych

przy testach związanych z Informacją Wzajemną. Razem mogą dawać one dobry wynik, czyli nie zostałyby wyrzucone przez $SelectFromModel()$.

Kolejna metoda zawarta w Pipeline'ach, czyli $SelectFpr()$ sprawdza korelację danych ze zmienną y_train. Wyrzucane są te zmienne, których korelacja jest najmniejsza. Pozbywamy się w ten sposób zmiennych, które stanowią szum wobec zmiennej celu y_train. Parametrem tej metody jest $alpha$, czyli próg odcięcia dla liczonego przez nią p-value. Im większe $alpha$, tym mniej szumu z danych usuwamy. Wartości parametru $alpha$ wykorzystane w walidacji krzyżowej w kolejnym etapie to $0,5$ oraz $0,8$, są to duże wartości, pozbywamy się niewielu zmiennych w ten sposób. Trzecią opcją przy walidacji będzie pominięcie tej procedury selekcyjnej.

Wszystkie wymienione powyżej metody zostały zaimplementowane w jednym Pipeline'ie. Wymienione wartości hiperparametrów umieszczono w siatce parametrów, która później zostanie przekazana do walidacji krzyżowej. Siatka ta została przedstawiona w tabeli 1.

Tabela 1: Parametry preprocessingu (Selekcja cech i korelacja).

Nazwa parametru	**Metoda**	**Szczegóły konfiguracji**
`selector`	**Metody entropijne**	`MutualInformation`, $k \in \{20, 40, 50, 60, 80, 90\}$
	SelectFromModel	LogisticRegression (L1, solver=`saga`, max_iter=5000)
		RandomForest (100 drzew, class_weight=`balanced`)
		RandomForest (50 drzew, threshold=`median`)
		RandomForest, 50 drzew, threshold=`mean`)
	Brak	`passthrough`
`correlation`	**SelectFpr**	Score func = `f_classif`, $\alpha = 0.8$
		Score func = `f_classif`, $\alpha = 0.5$
	Brak	`passthrough`

4 Budowa modeli

Zbudowano 8 różnych modeli. Proces wyboru hiperparametrów w każdym przypadku był jednakowy. Początkowo tworzono wstępny Pipeline z danym modelem, często bez żadnych ustawionych hiperparametrów. Jako preprocessing do każdego z takich Pipeline'ów dodano Pipeline opisany w sekcji 3. Przeprowadzono trzykrotką walidację krzyżową, przed podziałem dane były przesortowane. Stratyfikacja ze względu na podobny udział klasy 1 i 2 w zmiennej y_train nie jest konieczna. Przy użyciu funkcji $GridSearchCV$ wybrano optymalne parametry preprocessingu dla każdego modelu z osobna przy siatce parametrów przygotowania zbioru danych opisanej w sekcji 3. Następnie trenowano model na danych treningowych i odczytywano najbardziej optymalne parametry względem miary zrównoważonej dokładności. Stworzono nowe Pipeline'y, ale już z tymi optymalnymi parametrami preprocessingu, aby w kolejnym kroku móc przeprowadzić kroswalidację mającą na celu znalezienie najlepszych parametrów każdego z modeli. Podejście takie nie jest optymalne, ponieważ parametry preprocessingu i parametry modelu są ze sobą sprzężone. To, jaki preprocessing jest najlepszy, zależy od tego, jakiego modelu użyjemy. Optymalizując osobno, znajdujemy jedynie optimum lokalne, a nie globalne. Wykorzystano jednak takie podejście ze względu na dużą złożoność obliczeniową łącznego problemu optymalizacji hiperparametrów modelu i preprocessingu. Każdy z modeli miał inną siatkę hiperparametrów do optymalizacji. Ponownie przeprowadzono trzykrotną kroswalidację z przemieszaniem danych przed podziałem względem miary zrównoważonej dokładności. Optymalne hiperparametry preprocessingu i modelu wraz z ostateczną zrównoważoną dokładnością dodawano do odpowiednich list, z których na koniec powstała tabela.

Opiszemy teraz jakie modele zostały wykorzystane oraz względem jakiej siatki hiperparametrów były optymalizowane. W modelach, które opierają się na losowości ustawiono ziarno generatora na 333061 – hiperparametr $random_state$. Rozważane modele to:

1. **Regresja logistyczna** – Przy optymalizacji parametrów preprocessingu przyjęto hiperparametr $penalty = None$ oraz ustawiono maksymalną ilość iteracji na 1000. Siatka hiperparametrów do optymalizacji miary zrównoważonej dokładności znajduje się w tabeli 2, cały czas ilość iteracji ustawiona była na maksymalnie 1000.

Tabela 2: Siatka hiperparametrów dla modelu regresji logistycznej.

Strategia (Penalty)	**Solver**	**Parametr C**	**L1 Ratio**	**Class Weight**
Lasso ($L1$)	`liblinear`, `saga`	0.01, 0.1, 1, 10	—	`balanced`
Ridge ($L2$)	`lbfgs`, `liblinear`	0.01, 0.1, 1, 10, 100	—	`balanced`
ElasticNet	`saga`	0.01, 0.1, 1	0.2, 0.5, 0.8	`balanced`
Brak (None)	*Domyślny*	*Domyślny*	—	*Domyślny*

Optymalne hiperparametry przy regresji to:

- Preprocessing:

  - Selekcja cech – SelectKBest ($k = 20$, Mutual Info),
  - Korelacja – SelectFpr ($\alpha = 0.8$),
- Model:
  - Typ regularyzacji – $L2$ (Ridge),
  - Siła regularyzacji (C) – 10,
  - Solver – `lbfgs`,
  - Wagi klas – `balanced`,
- Zrównoważona dokładność najbardziej optymalnego modelu – 0.62. Niska wartość, najwidoczniej dane nie są liniowe.

2. **Support Vector Machine (SVM)** – Hiperparametry użyte przy optymalizacji preprocessingu to jądro nieliniowe 'rbf' ze względu na specyfikę danych oraz probability ustawione na True, aby dało się wyznaczyć *predict_proba*. Oprócz rozważanych powyżej hiperparametrów optymalizowane były parametry $C \in \{0.1, 0.95, 1, 1.05, 10, 100\}$ oraz $gamma \in \{scale, 0.1, 0.01\}$. C odpowiada za regularyzację, a *gamma* mówi o zasięgu pojedynczego punktu. Zakres parametru *gamma* nie jest optymalny dla modelu, jednak większa *gamma* może powodować przeuczenie się modelu. Optymalne hiperparametry przy SVM to:
   - Preprocessing:
     - Selekcja cech – SelectFromModel, RandomForest, 100 drzew,
     - Korelacja – `passthrough`,
   - Model:
     - Siła regularyzacji (C) – 1,
     - Gamma – 0.1,
   - Zrównoważona dokładność najbardziej optymalnego modelu – 0.92. Wartość byłaby większa gdyby nie próba uratowania przed przeuczeniem.

3. **Gradient Boosting** – Do preprocessingu użyto bazowego modelu. Siatka hiperparametrów znajduje się w tabeli 3. Optymalne hiperparametry to:
   - Preprocessing:
     - Selekcja cech – SelectFromModel, RandomForest, 100 drzew,
     - Korelacja – SelectFpr ($\alpha = 0.5$),
   - Model:

     'learning_rate': 0.05, 'max_depth': 3, 'n_estimators': 300, 'subsample': 0.8, 'min_samples_leaf': 5,
   - Zrównoważona dokładność najbardziej optymalnego modelu – 0.94. Wartość byłaby większa gdyby nie próba uratowania przed przeuczeniem.

Tabela 3: Siatka hiperparametrów dla modelu Gradient Boosting

Parametr	**Zakres**
Tempo uczenia (`learning_rate`)	0.01, 0.05
Liczba drzew (`n_estimators`)	200, 300
Max. głębokość (`max_depth`)	2, 3
Udział próbki (`subsample`)	0.8, 1.0
Min. próbek liść(`min_samples_leaf`)	5, 10

4. **K-najbliższych sąsiadów (KNN)** – Preprocessing na bazowym modelu, bez *random_state*, bo to model deterministyczny. Optymalizowane hiperparametry to $n_neighbors \in \{3, 5, 7, 11, 15\}$, czyli liczba branych pod uwagę najbliższych sąsiadów, $weights =' uniform'$ - taka sama siła głosów każdego z sąsiadów i $metric \in \{'euclidean', ' manhattan'\}$ - metryka, w której mierzone są odległości. Optymalne hiperparametry modelu KNN:
   - Preprocessing:
     - Selekcja cech – SelectFromModel, RandomForest, 100 drzew,
     - Korelacja – `passthrough`,
   - Model:
     - Liczba sąsiadów – 11,
     - Metryka – taksówkowa,
     - Wagi – jednakowe,
   - Zrównoważona dokładność najbardziej optymalnego modelu – 0.91. Wartość dobrana tak, żeby nie było znacznego przeuczenia.

5. **Quadratic Discriminant Analysis (QDA)** – – Preprocessing na modelu podstawowym bez hiperparametrów, w tym bez *random_state*, bo mamy model deterministyczny. Optymalizowany hiperparametr to parametr regularyzacji o wartościach w zbiorze $\{0.0, 0.01, 0.1, 0.5, 0.8\}$. Optymalne hiperparametry modelu QDA:
   - Preprocessing:
     - Selekcja cech – SelectFromModel, RandomForest, 100 drzew,
     - Korelacja – `passthrough`,

- Model: regularyzacja 0.1,
- Zrównoważona dokładność najbardziej optymalnego modelu – 0.72. Wartość niska, dane nie spełniają zależności kwadratowej.

6. **Random Forest** – Do preprocessingu wykorzystano podstawowy model. W tabeli 4 pokazano wykorzystane siatkę hiperparametrów. Optymalne parametry modelu RF to:

   - Preprocessing:
     - Selekcja cech: SelectKBest ($k = 20$, Mutual Info)
     - Korelacja: SelectFpr ($\alpha = 0.5$)
   - Model:

     'n_estimators': 200, 'max_depth': 10, 'min_samples_leaf': 10, 'max_features': 'sqrt',
   - Zrównoważona dokładność najbardziej optymalnego modelu – 0.93. Wartość wysoka, ale dobrana tak, by nie było przeuczenia.

Tabela 4: Siatka hiperparametrów dla modelu Random Forest

Parametr	**Zakres**
Liczba drzew	100, 200, 300
Max. głębokość	10, 20
Min. próbek liść	10, 12, 15
Max. cech	`sqrt`, `log2`

7. **Extra Trees** – Podstawowy model do preprocessingu. Tabela 5 zawiera optymalizowane hiperparametry. Optymalne wielkości dla modelu:

   - Preprocessing:
     - Selekcja cech – SelectFromModel, RandomForest, 100 drzew,
     - Korelacja – `passthrough`,
   - Model:

     'n_estimators': 300, 'max_depth': 15, 'min_samples_leaf': 6, 'max_features': 0.4,
   - Zrównoważona dokładność najbardziej optymalnego modelu – 0.95. Hiperparametry dobrane tak, by nie było przeuczenia.

Tabela 5: Siatka hiperparametrów dla modelu Extra Trees

Parametr	**Wartości**
Liczba drzew (`n_estimators`)	300, 500
Maksymalna głębokość (`max_depth`)	10, 15, 20
Min. próbek w liściu (`min_samples_leaf`)	6, 8, 10
Maks. liczba cech (`max_features`)	`sqrt`, 0.4

8. **Naive Bayes** – Bazowy model przy preprocessingu, bez parametru *random_state*, bo mamy model deterministyczny. Optymalizację prowadzono względem hiperparametru $var_smoothing \in \{10^{-9}, 10^{-8}, \ldots, 10^{-3}, 0.01, 0.1, 1.0\}$, który odpowiada za zmianę szerokości rozładu Gaussa. Optymalne parametry modelu NB:

   - Preprocessing: `passthrough` w obu przypadkach,
   - Model: Dopasowanie do rozkładu normalnego – 0.01,
   - Zrównoważona dokładność najbardziej optymalnego modelu – 0.64. Wartość niska, ponieważ nie jest to model kwadratowy.

5 Wybór najlepszego modelu

Ostateczne zbudowane modele docelowe były to dwa komitety modeli. A mianowicie Voting oraz Stacking z odpowiednimi wagami modeli opisanych w sekcji 4 i ich najlepszymi parametrami zarówno preprocessingu jak i samego modelu w każdym przypadku. Nie wykorzystano dwóch modeli dających najsłabsze wyniki zrównoważonej dokładności: regresji logistycznej i naiwnego Bayesa. Pozostałe modele weszły do komitetu z udziałami odpowiednio: Extra Trees-1/4, Gradient Boosting-5/24, SVM-1/6, KNN-1/6, Random Forest-1/6 oraz QDA-1/24. Wytrenowany w ten sposób model Voting daje zrównoważoną dokładność wynoszącą 0.943. Do modelu Stacking jako finalny estymator dodano regresję logistyczną i dało to zrównoważoną dokładność 0.929. Ostatecznie wybrano więc model Votingu. Sam model Extra Trees wykazywał większą wartość miary zrównoważonej dokładności, natomiast komitet jest bardziej optymalnym wyborem ze względu na to, że uwzględnia dużo modeli jednocześnie, być może więc wykazuje on lepszą wartość predykcji. Wektor prawdopodobieństw przynależności do klasy 1 dla obserwacji ze zbioru testowego dla modelu Voting został zapisany do pliku 333061_*artifical_prediction.txt*.

6 Podsumowanie

W ramach projektu przeprowadzono optymalizację ośmiu modeli klasyfikacyjnych, poprzedzoną zaawansowanym przygotowaniem danych z wykorzystaniem mechanizmu Pipeline i selekcji cech. Najlepsze rezultaty osiągnięto poprzez zastosowanie metod zespołowych, odrzucając słabsze estymatory liniowe na rzecz algorytmów opartych na drzewach decyzyjnych i SVM. Ostatecznie jako rozwiązanie finalne wybrano model Voting Classifier, który zapewnił najwyższą stabilność i zrównoważoną dokładność na poziomie $0,943$. Uzyskany wynik potwierdza, że agregacja zróżnicowanych modeli pozwoliła na najskuteczniejsze odwzorowanie nieliniowych zależności ukrytych w zbiorze danych.